Fachhochschule Nordwestschweiz
Hochschule für Technik

Software Architecture

Architecture Means

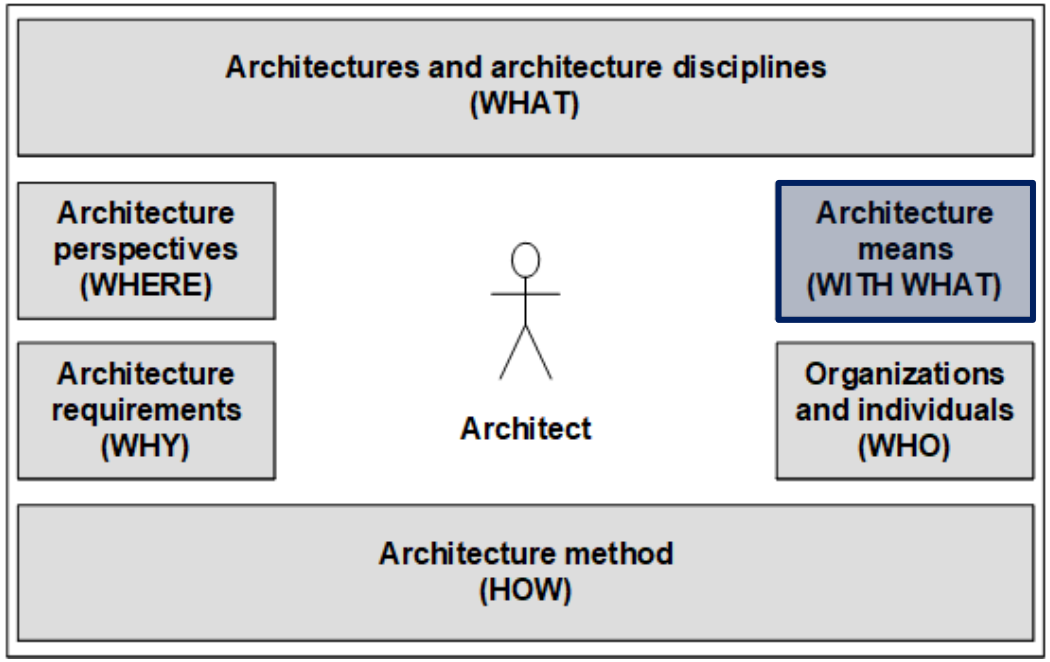
BSc

Ingo Arnold

Copyright Notice

Lecture Opening
Architecture Orientation Framework

Architecture **WITH WHAT discusses the broad range of architectural means architects use to plan, develop, and operate architectures**. Thus what we consider here is the architect's toolbox, if you wish. [Vogel, Arnold et al 2011].



Lecture Opening
Motivation

The software architect's toolbox is huge and offers a broad variety of tools. While no architect is an expert of all tools in that box, every architect should at least have an overview of its drawers, sub-drawers and sub-sub-drawers.

We will not be able to look at the entire toolbox in detail. In this deck I will only give a rough overview of the toolbox.

In the following units, we will then delve further into selected topics.

Lecture Opening
Learning Objectives

You …

- gained an overview of the architect's toolbox.
- understand the rough classification of the toolbox as well as its overall structure.
- know some of the architectural tools in the toolbox, so that you can orientate yourself in our following in-depth lectures.

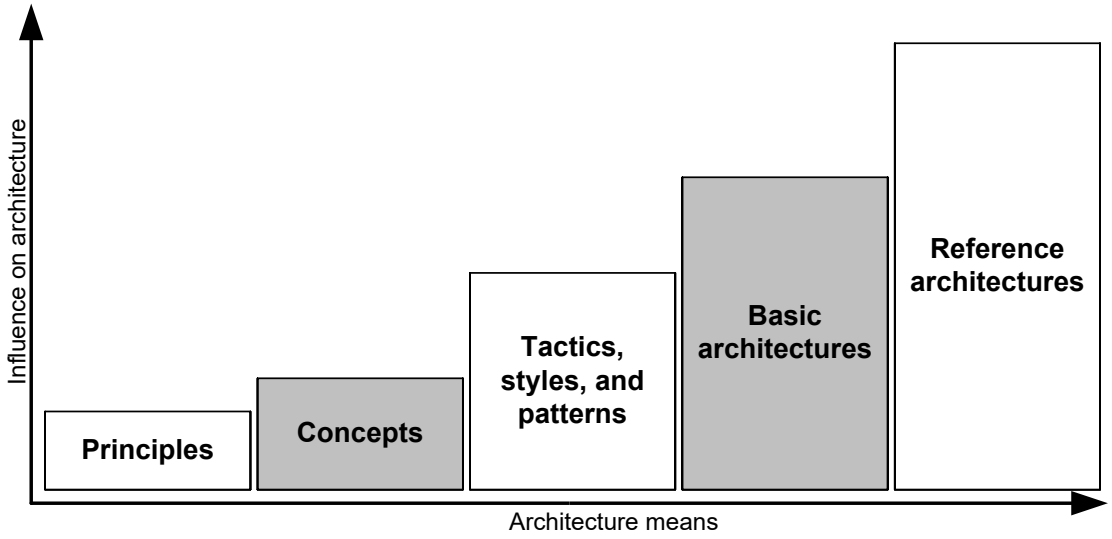
Lecture Agenda

- Architecture Means
- Architecture Principles
- Basic Architecture Concepts
- Architecture Tactics, Styles, and Patterns
- Basic Architectures
- Reference Architectures
- Domain-Driven Design

Architecture Means
Overview

Different types of architecture means influence the architecture under construction in different ways. The influence increases from the architecture principles right up to the reference architecture.

We will only dive deeper into a few selected principles, basic concepts, etc. in addition to the overview provided in this deck. For further classification, my book is recommended accordingly.



Lecture Agenda

- Architecture Means
- Architecture Principles
- Basic Architecture Concepts
- Architecture Tactics, Styles, and Patterns
- Basic Architectures
- Reference Architectures
- Domain-Driven Design

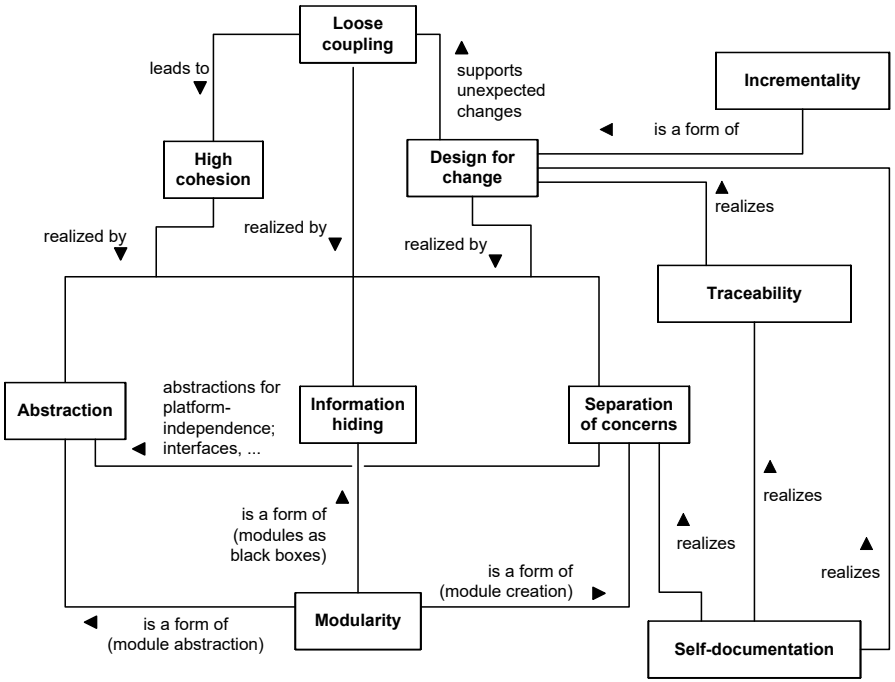
Architecture Principles
Overview

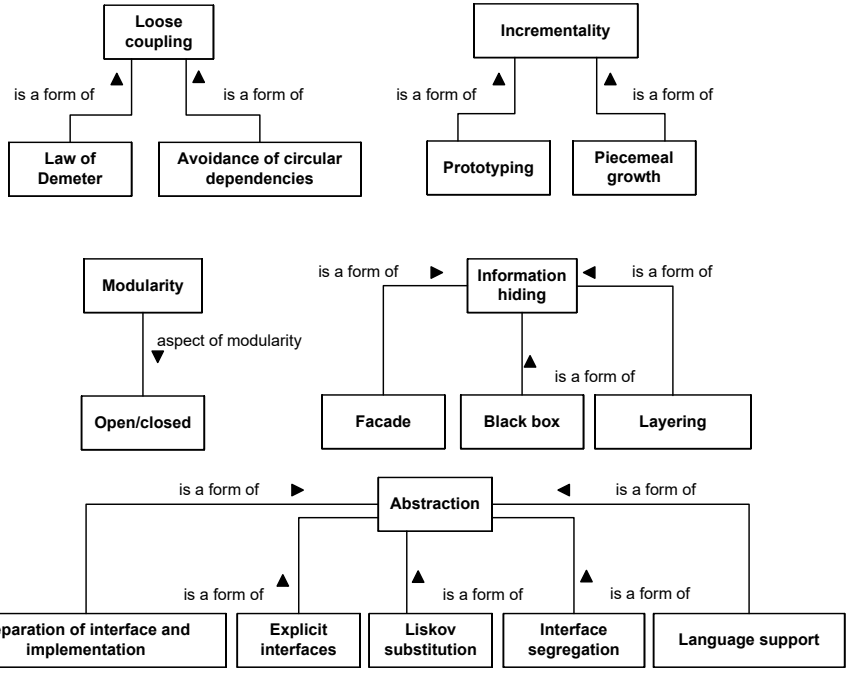
Architecture principles are proven guidelines that you should apply when developing or modifying an architecture. However, they say nothing about how you should apply these principles in concrete cases.

It is difficult to say that an architecture is good or bad in itself (it merely fulfils its conditions more or less well). However, if you take general principles into account when designing a software architecture, you tend to get a good architecture.

Architecture Principles
Overview

Selected principles and their relationships.





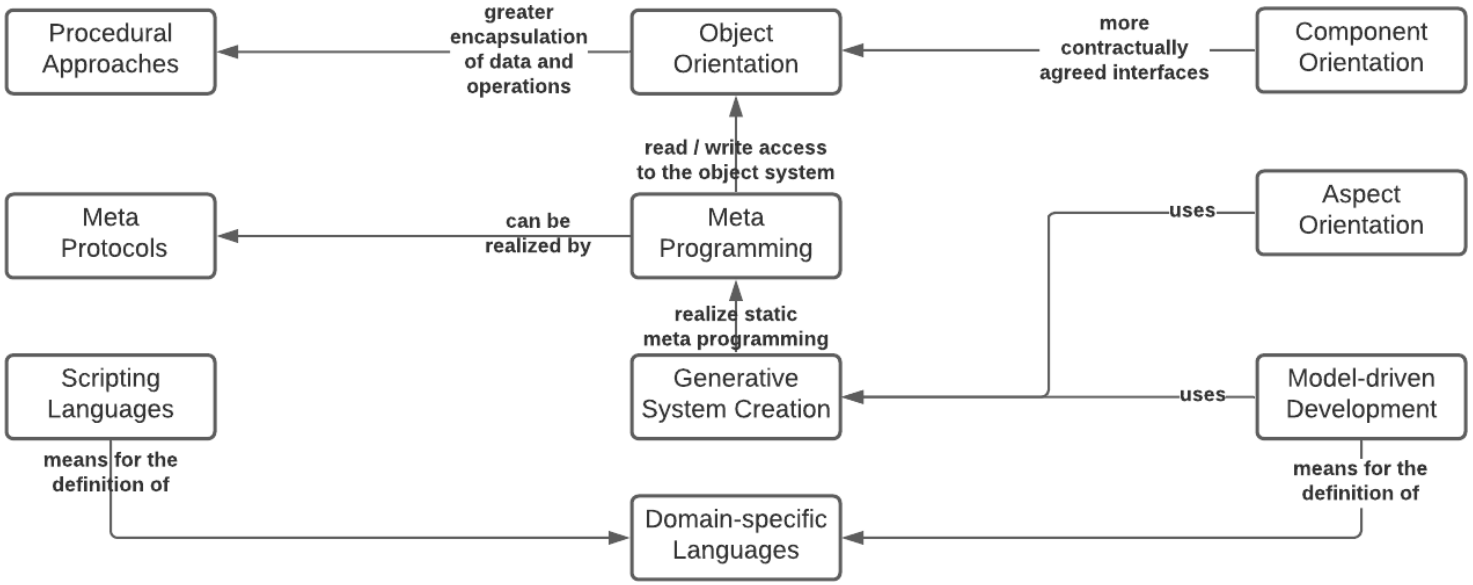
Lecture Agenda

- Architecture Means
- Architecture Principles
- Basic Architecture Concepts
- Architecture Tactics, Styles, and Patterns
- Basic Architectures
- Reference Architectures
- Domain-Driven Design

Basic Architecture Concepts
Overview

Basic Concepts is what the name implies: essential paradigms and concepts of architecture.

In my book, this section starts with simple procedural approaches and then gradually moves to more comprehensive approaches such as aspect orientation, component orientation, and model-driven architecture.



Basic Architecture Concepts
Overview

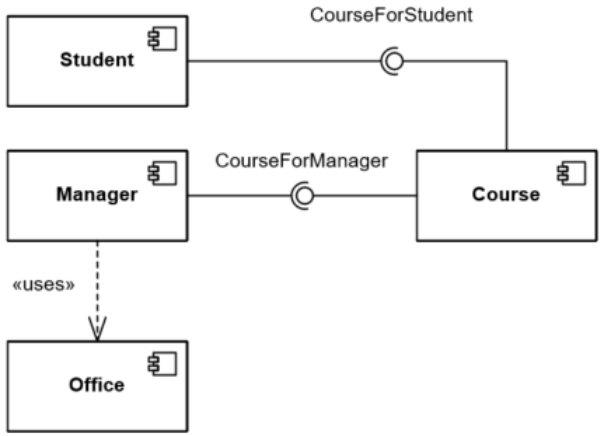
Brief overview of basic architecture concepts.

Procedural Approaches	Procedures are a traditional and widely used means of structuring architectures. They allow the decomposition of a complex algorithm into repeatable individual algorithms. Procedures implement the principle of separation of concerns, since they make it possible to decompose a complex algorithmic problem into simpler subproblems
Object Orientation	Object orientation is based on the idea of bundling data processed by a set of related procedures together with these procedures. The procedures – called operations or methods in object orientation – can process their data exclusively. In this way, object orientation attempts to directly implement the principle of information hiding and the principle of modularity

Basic Architecture Concepts
Overview

Brief overview of basic architecture concepts.

Component Orientation	Components are reusable, self-contained building blocks of an architecture. The component orientation arose from the problem that objects implement the modularity principle, but are often too small to be used as reusable units. In addition one desires from a reusable system building block often further characteristics, like the implementation of non-functional requirements, or the simultaneous provision of several identical component instances for increasing the scalability. Component platform examples are DLLs, JavaBeans, ActiveX, JEE components like EJBs, .NET components, CORBA components



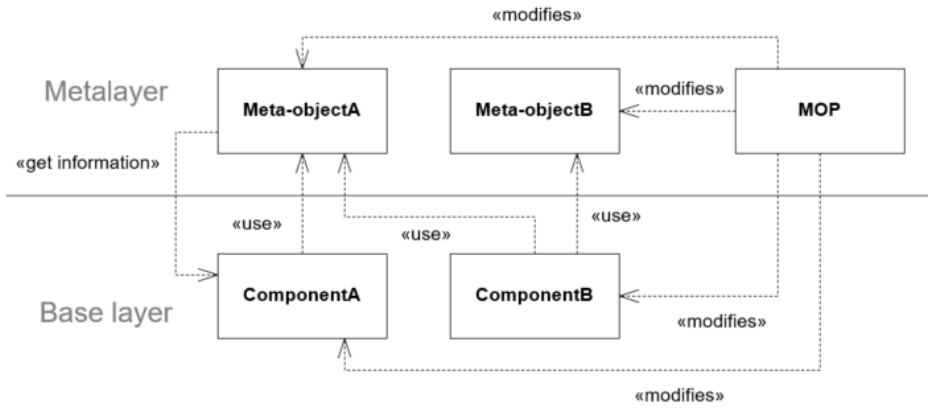
Basic Architecture Concepts
Overview

Brief overview of basic architecture concepts.

Meta Programming

Many programming languages distinguish between the program as a set of executable instructions and the data that the program works with. Actually, the programs themselves are also data. So in metaprogramming, programs are treated as data. Programs that use other programs (or themselves) as data are usually called metaprograms.

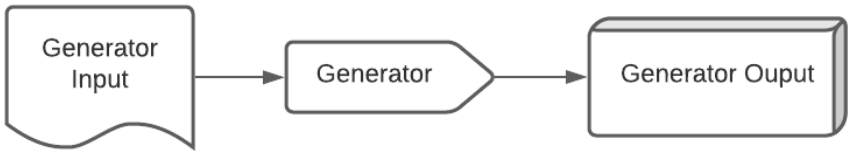
Meta programming techniques and technologies are reflection or introspection capabilities (e.g., java reflection API), macro mechanisms (e.g., C++ pre-processor changes the program before it is sent to the C++ compiler)



Basic Architecture Concepts
Overview

Brief overview of basic architecture concepts.

Generative System Creation	If we look beyond software development, we see that in other engineering disciplines, processes are always automated when certain recurring tasks have to be performed in a similar way. The use of generative technologies in software development aims to adapt the degree of automation in the creation of software to that of other engineering disciplines. Examples of generators are XSLT, annotation-based Javadoc generators, IDL generators, “inline” keyword in C/C++



Basic Architecture Concepts
Overview

Brief overview of basic architecture concepts.

Model-Driven Development	Model-driven software development (MDD) is when models are not only used for documentation purposes, but are central artifacts of an executable system. A special case of the generative system creation discussed above. Generator input is a system model while the generator output is the physical equivalent of the initially modelled system.
Aspect Orientation	Aspect orientation avoids spreading so-called crosscutting concerns across the code or design. These are concerns that are general or go beyond the application logic. Security is an example of such a concern.

Basic Architecture Concepts
Overview

Brief overview of basic architecture concepts.

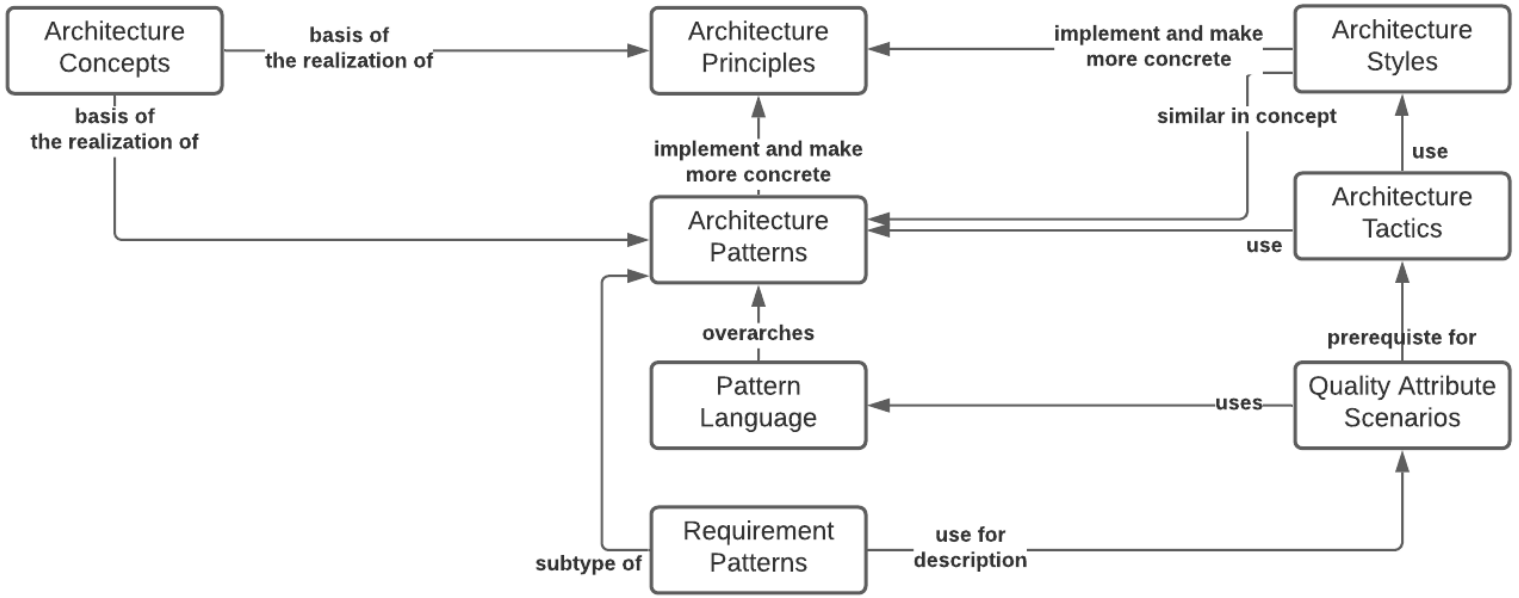
Scripting Languages and Platform Object Models	Scripting languages are originally programming languages for controlling software systems. They typically use a two-language approach. The core system is implemented in a programming language other than the scripting language, and the scripting language only handles the composition of the system components in an executable system. Examples are JavaScript and the HTML DOM, or ABAP in SAP systems.

Lecture Agenda

- Architecture Means
- Architecture Principles
- Basic Architecture Concepts
- **Architecture Tactics, Styles, and Patterns**
- Basic Architectures
- Reference Architectures
- Domain-Driven Design

Architecture Tactics, Styles, and Patterns
Overview

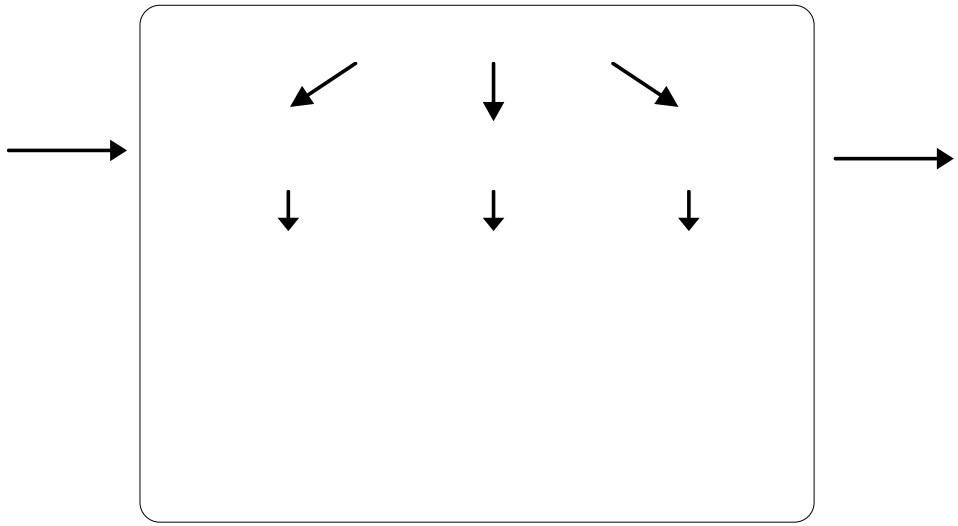
In my book this section copes with architectural tactics, styles, and patterns. These three means have in common that they describe principle solutions to certain recurring problems in architectural design.



Architecture Tactics, Styles, and Patterns
Overview

Brief overview of tactics, styles, and patterns.

Architecture Tactic	Architectural tactics provide guidelines for implementing a wide variety of quality features of the system being designed and its architecture. So, in principle, an architectural tactic helps you to get a first idea about a design problem. You can then develop this idea further and, for example, also use styles and patterns as further tools.

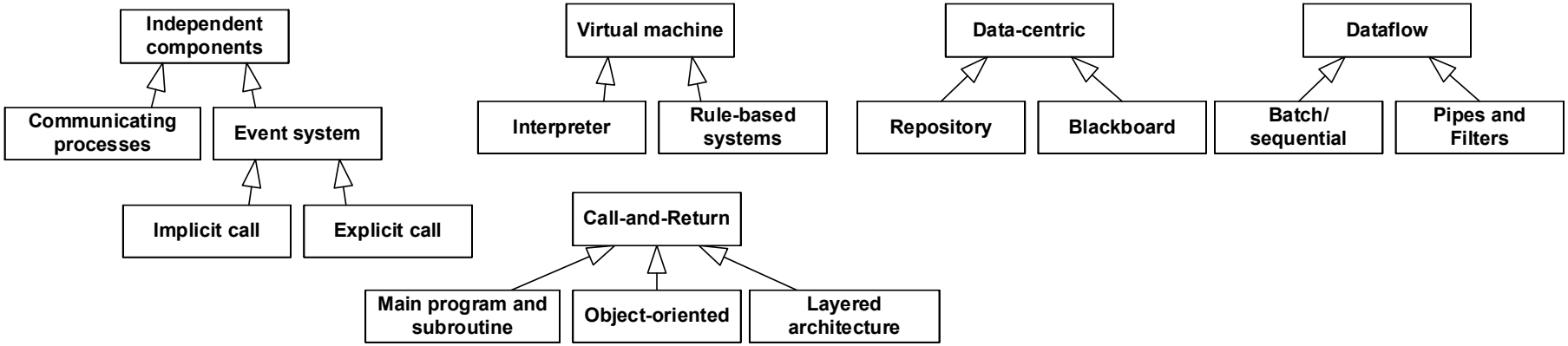


Architecture Tactics, Styles, and Patterns
Overview

Brief overview of tactics, styles, and patterns.

Architecture Style

An architectural style is a pattern of structural organization of a family of systems. For them, an architectural style consists of the following elements: a set of building blocks that perform certain functions at runtime. A topological arrangement of these building blocks. A set of connectors that govern communication and coordination among the building blocks. A set of semantic restrictions that specify how building blocks and connectors can be connected to each other. An architecture style primarily reflects the fundamental structure of a software system and its properties. You can therefore use a style to categorize architectures.



example

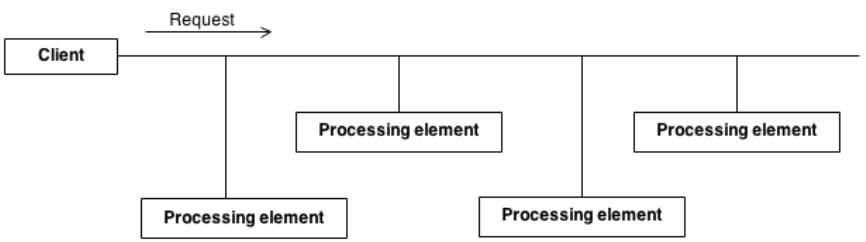
Architecture Tactics, Styles, and Patterns
Overview

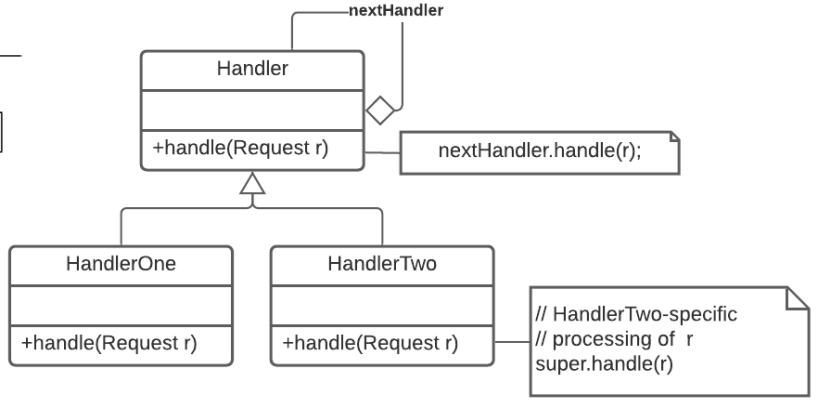
Brief overview of tactics, styles, and patterns.

Architecture Pattern

In the context of software architecture, both design patterns and architecture patterns are very important. They have in common that they usually represent structural, technical solutions. In general, design patterns describe more specific design solutions that have a local impact, while architecture patterns describe more system structures that have an impact on the entire system.

Example **chain of responsibility** pattern



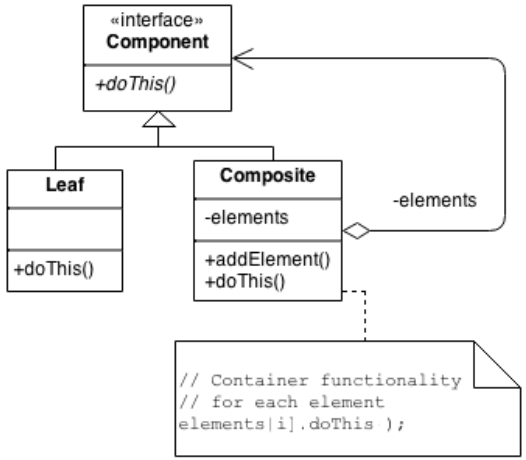


example

Architecture Tactics, Styles, and Patterns
Overview

Brief overview of tactics, styles, and patterns.

Architecture Pattern	Example **composite** pattern

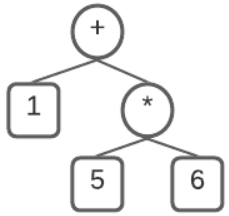


example

Architecture Tactics, Styles, and Patterns
Overview

Brief overview of tactics, styles, and patterns.

Architecture Pattern	Apply the composite pattern for a simple expression interpreter

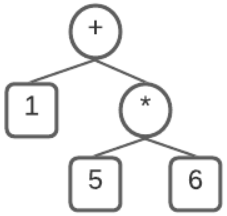


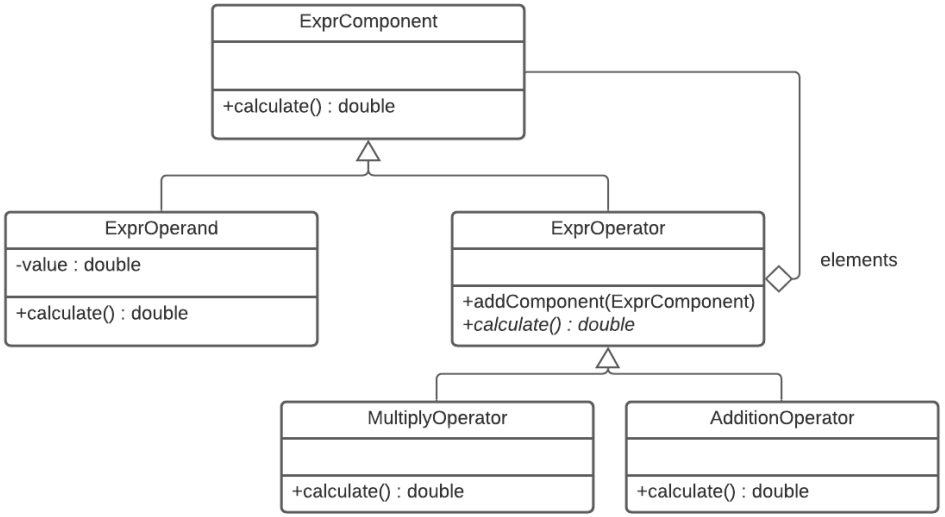
example

Architecture Tactics, Styles, and Patterns
Overview

Brief overview of tactics, styles, and patterns.

Architecture Pattern Apply the composite pattern for a simple expression interpreter





Architecture Tactics, Styles, and Patterns
Overview

Brief overview of tactics, styles, and patterns.

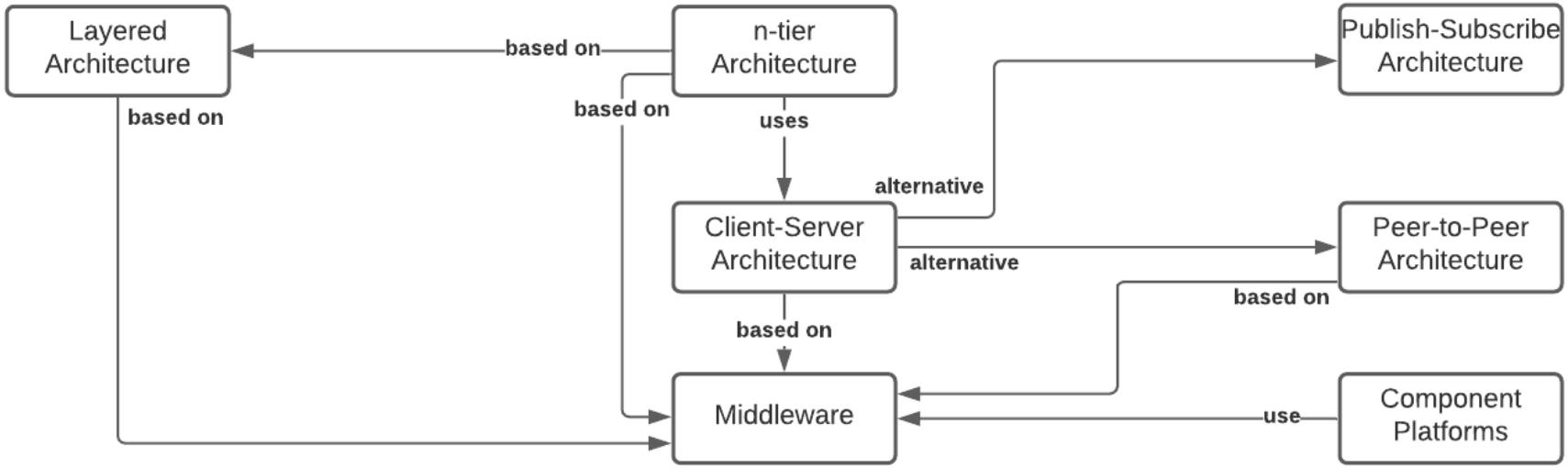
Architecture Pattern Language	A pattern language is a collection of semantically related patterns that provide solution principles for problems in a particular context. The focus of pattern languages is particularly on the relationships of the patterns in the language. Specifically, this means that the pattern descriptions are highly integrated – for example, in that the context of one pattern picks up another pattern and that particular emphasis is placed on describing the pattern interactions
Requirements Pattern	A requirements pattern is a methodical tool that you can use to systematically develop "good" requirements. Like architecture patterns, requirements patterns refer to visibly recurring issues for which generally valid solution proposals can be formulated. A requirements pattern can thus take the form of a template for concrete requirements of a specific requirements category.

Lecture Agenda

- Architecture Means
- Architecture Principles
- Basic Architecture Concepts
- Architecture Tactics, Styles, and Patterns
- Basic Architectures
- Reference Architectures
- Domain-Driven Design

Basic Architectures
Overview

This section of my book discusses foundational architecture approaches that are adopted in many systems. Basic architectures use the various architectural means discussed earlier. They therefore represent a more concrete architectural guideline that you can use to structure entire systems.



Basic Architectures
Overview

Brief overview of basic architectures.

Layered Architecture	A fundamental issue of architectures that affects both functional and non-functional requirements is the partitioning of a system into groups of similar functions or responsibilities. The layers style describe a typical solution. One applies layers in situations where one group of building blocks depends on another group of building blocks to perform its function, and that group in turn depends on another group of building blocks, and so on. The layered pattern states that each layer groups a number of building blocks and the layer above it provides services through interfaces. In each layer, the building blocks of that layer can freely interact with each other. Between two layers, communication may only take place via the specified interfaces.
N-tier Architecture	A tier is a means of structuring the distribution of software building blocks on hardware building blocks. A tier contains interrelated system building blocks (software and hardware building blocks) that are interconnected via a network. For example, the classic client-server model based on a two-tier architecture.

Basic Architectures
Overview

Brief overview of basic architectures.

Client-Server Architecture	In modern software architecture, client-server operation is of great importance. Here, the user runs application programs (clients) on his computer and these programs access the resources of the server. The resources are centrally managed, shared and made available. The client-server model is based on a simple request-response scheme. This means that files no longer have to be transferred as a whole, but can be requested specifically.
Publish Subscribe Architecture	Publish-subscribe addresses the problem of informing a set of clients about runtime events. Publish-subscribe allows the event consumer to register for specific event types. When such an event occurs, the event producer informs all registered consumers - for example, via a publish/subscribe system. The publish-subscribe system thus decouples the producer and the consumer of events.

Basic Architectures
Overview

Brief overview of basic architectures.

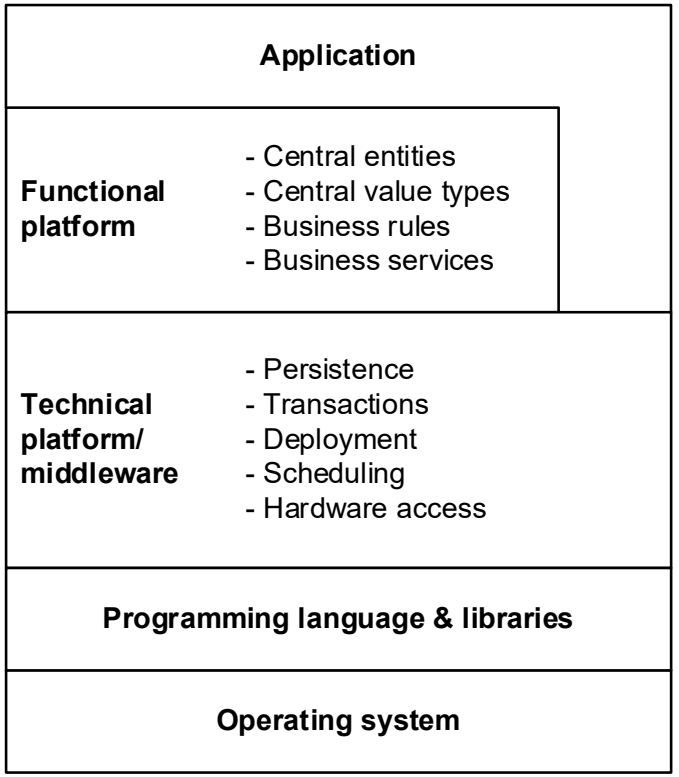
Peer-2-Peer Architecture	Peer-to-peer (P2P) uses a number of equal peers. In the pure P2P architecture structure, there is no central server. Each peer can offer and take advantage of services on the network. The entire status of the system is distributed among the peers. Internally, P2P systems are partly implemented with standard middleware, but the user of a P2P system is not aware of this. In the P2P architecture, one can add or remove a service at any time. Clients must therefore find out which services are currently available before using a service.
Middleware	Middleware establishes a platform that provides application services for all aspects of distribution, such as distributed invocation, efficient access to the network, transactions, messaging and directory services, time services, security services, and more. Middleware establishes a type of network operating system (NOS) that makes the complexities of a distributed environment accessible to software developers efficiently and effectively.

Basic Architectures
Overview

Brief overview of basic architectures.

Component Platforms

Typical examples of component platforms in the enterprise environment are JEE or .NET. These component platforms are based on the separation of technical concerns and functional concerns for an information system. Examples of technical concerns in the enterprise environment include distribution, security, persistence, transactions, concurrency, and resource management.
In other environments, such as embedded systems, other technical concerns may be important.

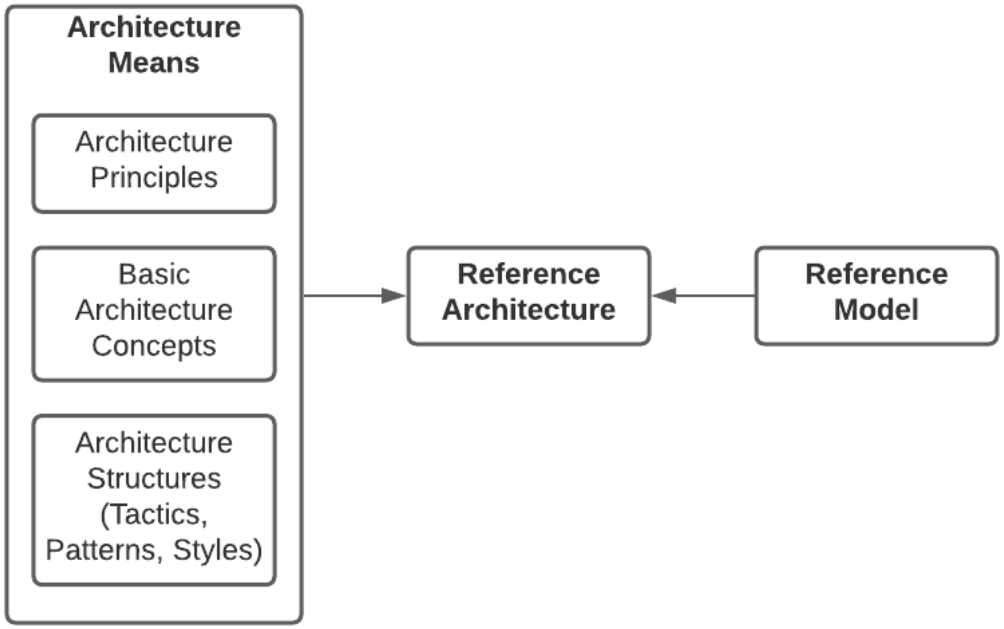


Lecture Agenda

- Architecture Means
- Architecture Principles
- Basic Architecture Concepts
- Architecture Tactics, Styles, and Patterns
- Basic Architectures
- Reference Architectures
- Domain-Driven Design

Reference Architectures
Overview

Reference architectures combine general architectural knowledge and experience with specific requirements for a coherent architectural solution for a particular problem domain. They document the structures of the system, the essential system building blocks, their responsibilities and their interaction.



Reference Architectures
IBM reference architecture examples

IBM offers a range of reference architectures for selected topics.

Data fabric

Unlock the value of all of your accessible data to gain valuable insights. Discover, govern, and secure your data.

Business automation

Automate your business processes to increase productivity and performance. Incorporate AI models to create new workflows that take advantage of your data.

Observability

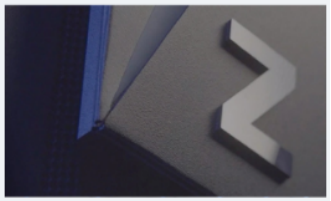
Bring your operational information into one management platform to proactively predict problems and build automated responses to keep downtime to a minimum.

Security

Build the best possible defense to keep your enterprise safe and secure. Protect your enterprise from perpetrators and recover rapidly from security incidents.

Regulated workloads

Adopt a cloud platform that complies with industry regulations while delivering the benefits of cloud native applications and development.



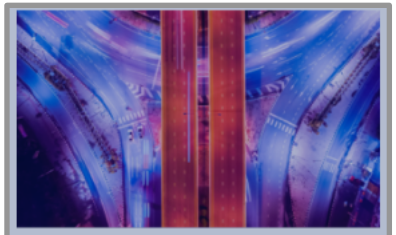
IBM Z

Integrate your IBM Z systems of record into your hybrid cloud transformation strategy to get the best of both worlds.

https://www.ibm.com/cloud/architecture/

Reference Architectures
IBM reference architecture examples

One of these reference architectures IBM calls **business automation**.



Workflow management

Automate your business workflows to increase agility, visibility, and consistency across your business processes.

Content services

Share and govern unstructured content so that you can better engage customers, automate business processes, and enhance collaboration.

Decision management

Automate repeatable decisions and decisions that are based on policies and business rules to help an organization meet its goals.

Robotic process automation

Use robotic process automation (RPA) and digital workers to automate mundane tasks so that employees can focus on higher value work.

Document processing

Capture and extract data from documents that you can use to optimize your business processes.

Operational intelligence

Use operational intelligence to analyze data and gather insights that you can use to improve your business

Integration

Integrate cloud applications across hybrid and multicloud environments.

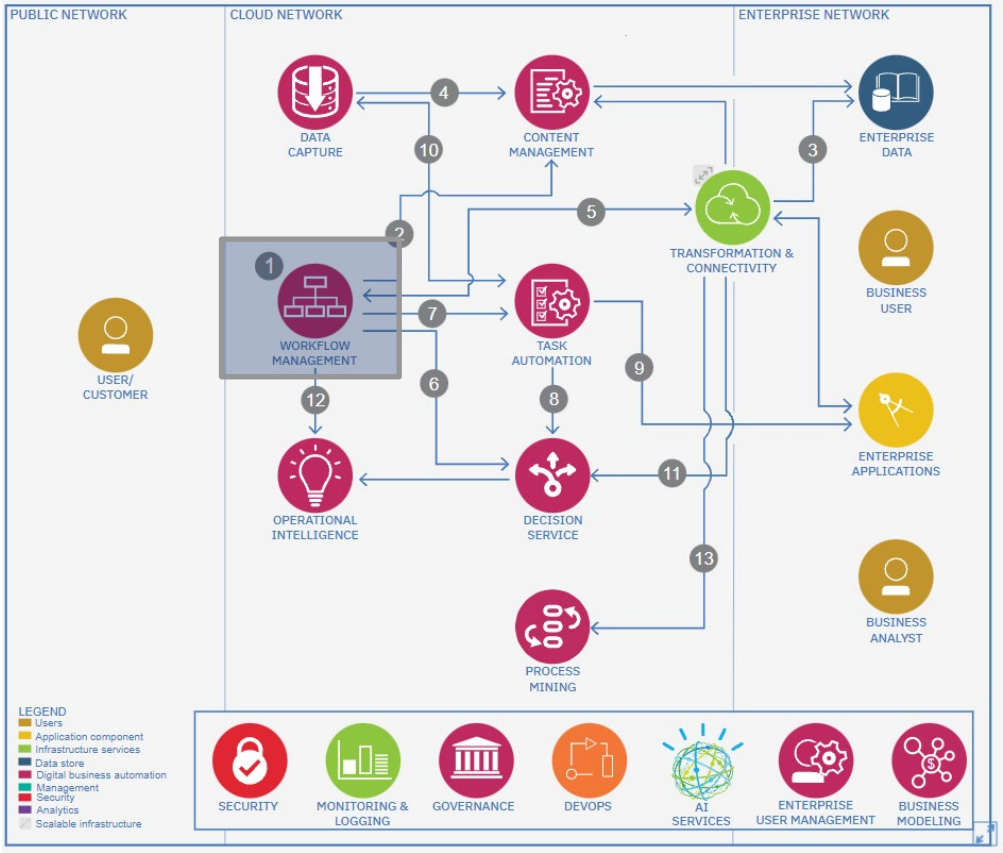
Event driven

Develop and extend reactive applications by using CQRS and event sourcing.

https://www.ibm.com/cloud/architecture/technical-decision-points/business-automation

Reference Architectures
IBM reference architecture examples

The **business automation** reference architecture

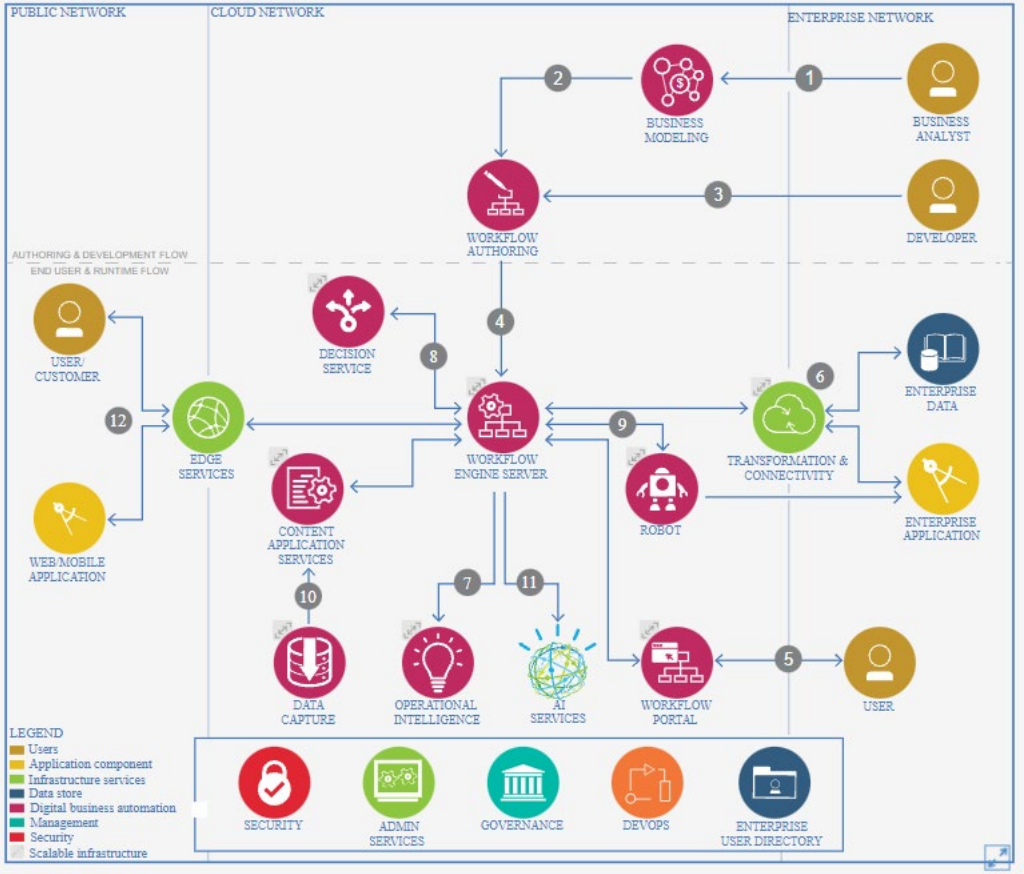


https://www.ibm.com/cloud/architecture/architectures/dba-architecture/reference-architecture

Reference Architectures

IBM reference architecture examples

Workflow management reference architecture



https://www.ibm.com/cloud/architecture/architectures/workflowDomain/reference-architecture

Lecture Agenda

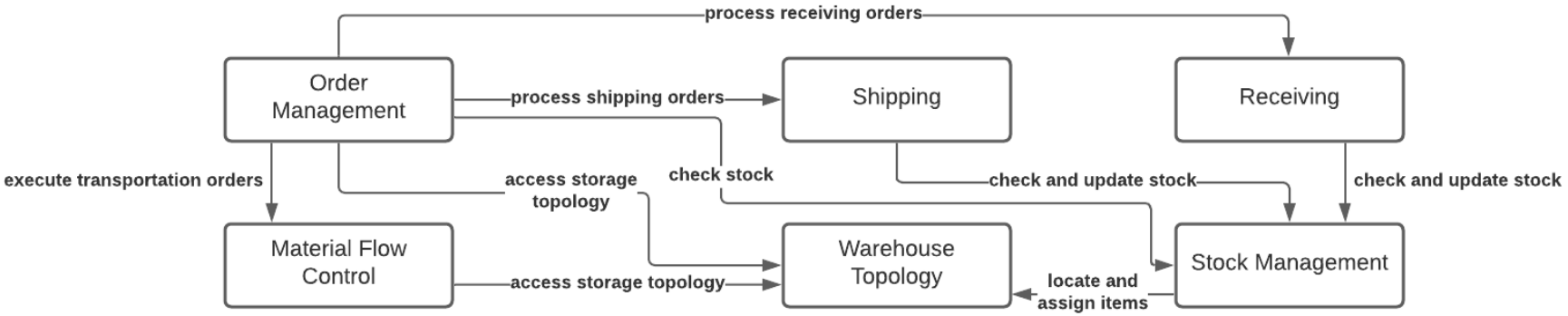
- Architecture Means
- Architecture Principles
- Basic Architecture Concepts
- Architecture Tactics, Styles, and Patterns
- Basic Architectures
- Reference Architectures
- Domain-Driven Design

Domain-Driven Design
Domain Model

A **Domain Model establishes** a **common language** to **consistently name** and understand **key terms**, **concepts** and their **relationships** in a given **business domain**.

Rather, it is a **living working tool** that is meant to **increase efficiency** and **effectiveness** of **communication** across a broad **variety of stakeholders** and to **evolve iteratively** as a **journey progresses**.

Illustrative example of a simplified *Domain Model* for warehouse management:



Domain-Driven Design
Domain Model

A **Domain Model** is **not** a **technical data model** or **solution design** (despite being mis-concepted as such sometimes).

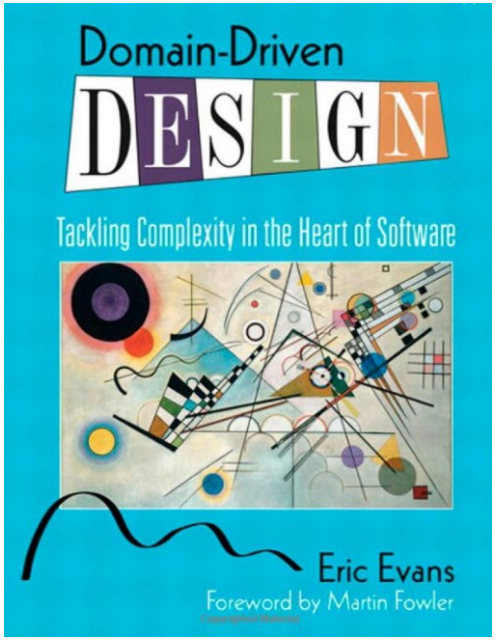
A **Domain Model** does **not define any system architecture** or **implementation** and does **not claim** to be **final and complete** at any given point in time.

The **aim** of a **Domain Model** is to **detect** and **avoid misunderstandings**, facilitate communication and create a **Ubiquitous Language** for sound technical and business decisions – especially in interdisciplinary teams with different perspectives on the domain.

Domain-Driven Design
Domain Model

A **Domain Model** provides the **initial step** in **transforming requirements** into a **sustainable software architecture** and implementation.

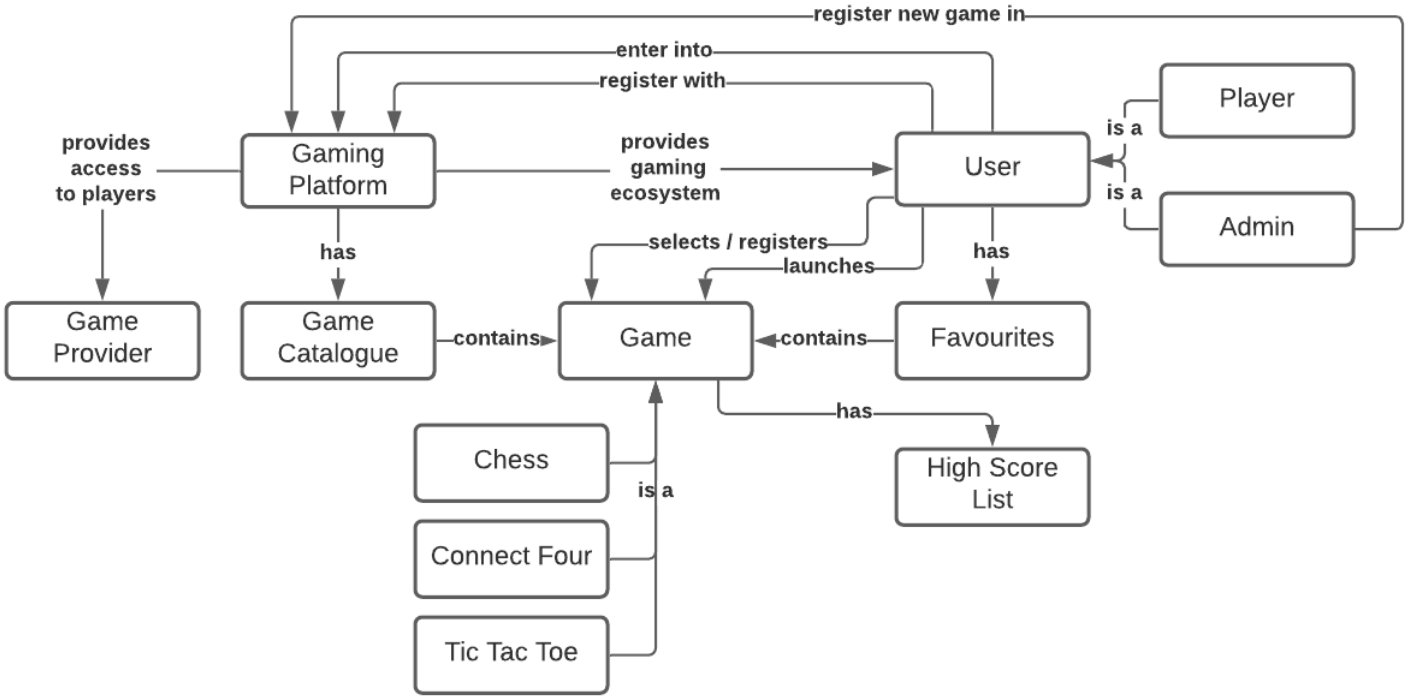
In **Eric Evans** book **Domain-Driven Design** a **Domain Model** is a **first** (and yet technology-agnostic) **step** in **developing** a **sound software design**.

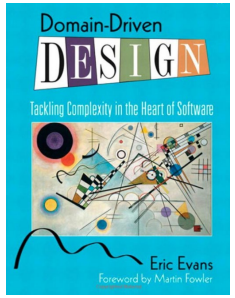


example

Domain-Driven Design
Domain Model

We could have created a **Domain Model** for the **gaming platform**.

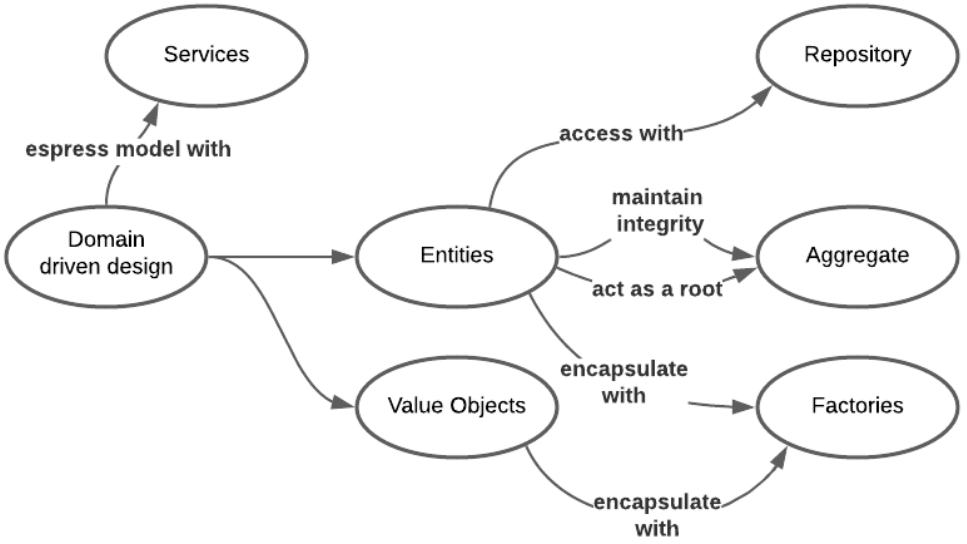


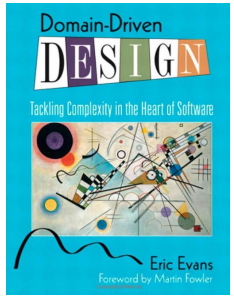


Domain-Driven Design
Domain Model – Domain-driven Design

Domain-driven design distinguishes the following components of the *Domain Model*:

- Entities (reference objects)
- Value objects
- Aggregates
- Associations
- Service objects (services)
- Domain events
- Modules (modules, packages)
- Factories
- Repositories





Domain-Driven Design
Domain Model – Domain-driven Design

Domain-driven design distinguishes the following components of the *Domain Model*:

- **Entities (reference objects)**

  Objects of the model, which are not defined by their properties, but by their identity. For example, a person is usually represented as an entity. Thus, a person remains the same person if his or her properties change; and he or she is different from another person even if the latter has the same properties. Entities are often modelled using unique identifiers.

- **Value objects**

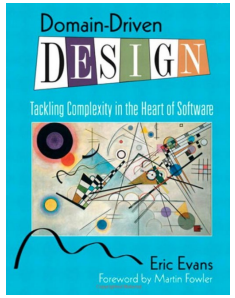
  Objects of the model that do not have or require a conceptual identity and are thus defined solely by their properties. Value objects are usually modelled as immutable objects, which makes them reusable (cf. Flyweight) and distributable.

- **Service objects (services)**

  In domain-driven design, functionalities that represent an important concept of the domain model and conceptually belong to several objects of the domain model are modelled as independent service objects. Service objects are usually stateless and therefore reusable classes without associations, with methods that correspond to the provided functionalities. These methods are passed the value objects and entities necessary to process the functionality.

- **Aggregates**

  Aggregates are combinations of entities and value objects and their associations to a common transactional unit. Aggregates define exactly one entity as the only access to the entire aggregate. All other entities and value objects may not be statically referenced from outside. This guarantees that all invariants of the aggregate and the individual components of the aggregate can be ensured.



Domain-Driven Design
Domain Model – Domain-driven Design

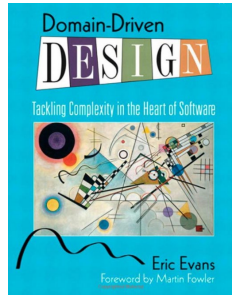
Domain-driven design distinguishes the following components of the *Domain Model*:

- **Factories**

  Factories are used to outsource the creation of domain objects to special factory objects. This is useful if either the creation is complex (and, for example, requires associations that the subject object itself no longer needs) or the specific creation of the subject objects should be able to be exchanged at runtime. Factories are usually implemented by generating design patterns such as abstract factory, factory method or builder.

- **Repositories**

  Repositories abstract the persistence and search of business objects. Repositories separate the technical infrastructure and all access mechanisms to it from the business logic layer. For all business objects that are loaded via the infrastructure layer, a repository class is provided that encapsulates the loading and search technologies used from the outside. The repositories themselves are part of the domain model and thus part of the business logic layer. They are the only ones that access the objects of the infrastructure layer, which are usually implemented using the design patterns Data Access Objects, Query Objects or Metadata Mapping Layers.



Domain-Driven Design
Domain Model – Domain-driven Design

Domain-driven design distinguishes the following components of the *Domain Model*:

- **Domain events**

  Domain events are objects that describe complex, possibly dynamically changing, actions of the domain model that cause one or more actions or changes in the domain objects. Domain events also enable the modelling of distributed systems. The individual subsystems communicate exclusively via domain-oriented events, so they are strongly decoupled and the entire system is thus more maintainable and scalable.[9].

- **Modules (modules, packages)**

  Modules divide the domain model into functional (not technical) components. They are characterized by strong internal cohesion and low coupling between the modules.

- **Associations**

  Associations, as defined in UML, are relationships between two or more objects of the domain model. Here, not only static relationships defined by references are considered, but also dynamic relationships that arise, for example, only through the processing of SQL queries.

Wrap-Up

- **Architecture Means**
  - **Definition**: All methodological tools, proven design blueprints, and mechanisms architects use to plan, develop, and operate architectures.
  - **Influence increases** from **principles** → **concepts** → **tactics/styles/patterns** → **basic architecture** → **reference architectures**.
  - Acts as the **architect's toolbox**, linking theoretical foundations to practical design methods.
- **Architecture Principles**
  - **Definition**: **Proven**, **experience-based guidelines** for developing or evolving architectures.
  - They **do not prescribe *how*** to **implement**, but help achieve good architectural quality.
  - **Typical principles** and relations:
    - Modularity (black-box, open/closed, information hiding)
    - Abstraction and Separation of Concerns
    - Loose coupling / High cohesion
    - Incrementality / Design for change
    - Explicit interfaces / Interface segregation
    - Traceability / Self-documentation
  - **Principles** form the **foundation** for **robust**, **maintainable**, and **evolvable architectures**.

Wrap-Up

- **Basic Architecture Concepts**
  - **Purpose**: **Fundamental paradigms shaping software architectures**.
  - **Main concepts**
    - **Procedural approaches** – break complex algorithms into manageable parts (separation of concerns).
    - **Object Orientation (OO)** – encapsulates data + behavior (information hiding, modularity).
    - **Component Orientation** – larger, reusable blocks combining functionality and non-functional aspects.
    - **Meta Programming** – programs treat code as data (reflection, macros).
    - **Generative Systems** – automate repetitive development tasks (e.g., code generators).
    - **Model-driven Development (MDD)** – models as primary artifacts; generators produce code.
    - **Aspect Orientation** – manages cross-cutting concerns (e.g., security, logging).
    - **Scripting languages / platform object models** – combine configuration and automation (e.g., JavaScript/HTML DOM).
- **Architecture Tactics, Styles, and Patterns**
  - **Common goal**: **Provide reusable solution approaches** for **recurring architectural problems**.
  - **Tactics**
    - **Target specific quality attributes** (performance, security, availability, modifiability).
    - **Serve** as **initial design heuristics before applying styles** or **patterns**.

Wrap-Up

- **Architecture Tactics, Styles, and Patterns**
  - **Styles (**aka: **Architecture Patterns)**
    - **Define structural organization** and **relationships** (dynamic and static) between **components**.
    - **Examples** are Layered Architecture, Client-Server, Pipes & Filters, Shared Repository.
  - **Patterns and Pattern Languages**
    - **Capture proven design** or architecture solutions **at different granularities**.
      - **Design patterns**: local and fine-grained, at class-level (e.g., Composite, Chain of Responsibility).
      - **Architecture patterns**: system-wide and coarse-grained, at component-level (aka: Styles).
      - **Requirements patterns** provide blueprints for recurring types of requirement (e.g., reporting, printing, authN & authZ, documentation).
    - **Pattern languages interlink related patterns** into **coherent pattern** "**families**".
- **Basic Architectures**
  - **Definition**: **Foundational architecture structures** used **across systems**.
  - **Combine multiple architecture means into practical frameworks**.
  - **Examples**:
    - **Middleware** – abstracts distributed systems (transactions, messaging, security).
    - **Component Platforms** – enterprise frameworks (e.g., JEE, .NET) that separate functional from technical concerns.

Wrap-Up

- **Reference Architectures**
  - **Definition**: **Combine general architectural knowledge** with **domain-specific requirements** to **guide holistic** (i.e., **socio-technical**) **solution design**.
  - **Serve** as **templates** or **blueprints** that **define key components**, **interactions**, and **best practices**.
  - **Example**: **IBM Cloud Reference Architectures** (e.g., Business Automation, Workflow Management).
  - **Goal**: **Ensure consistency**, **reusability**, and **faster architecture development** across similar problem domains **for socio-technical systems**.

Wrap-Up

- **Domain-Driven Design**
  - **Purpose**: **Align software structures** closely **with** the **business domain**.
  - **Domain Model**: **shared language** (aka: **Ubiquitous Language**) **representing key concepts** and **their relationships**.
    - **Improves communication** and **understanding between Business** and **IT**.
    - **Not** a **technical data model**; evolves iteratively.
  - **Core elements**:
    - **Entities** (aka: **Reference Objects**) – identity-based objects (e.g., Person).
    - **Value Objects** – immutable, property-based objects (e.g., Address).
    - **Aggregates** – consistency boundaries combining entities/value objects.
    - **Repositories** – abstract persistence, encapsulate data access.
    - **Factories** – control creation of complex domain objects.
    - **Services** – stateless operations spanning multiple objects.
    - **Domain Events** – capture important occurrences in the business domain.
    - **Modules** – cohesive functional groupings.
    - **Associations** – relationships between domain objects.
  - **DDD bridges business understanding** and **software architecture structures** – the **first step from domain logic** to **software design**.

Questions

Bibliography
Lecture

[Vogel, Arnold et al 2011]
Vogel, Oliver; Arnold, Ingo; Chugtai, Arif; Kehrer, Timo, *Software Architecture: A Comprehensive Framework and Guide for Practitioners*, Springer Science and Business Media, Berlin Heidelberg, 2011